FinWave

FinTech Financial Inclusion Platform for Migrant Workers

DESIGN MODEL DOCUMENT

Team 20 | MIT Academy of Engineering

Hackathon 2026 | 22 May 2026

Version 1.0.0 | Final Submission

1. Process Model — Agile (Scrum-Inspired)

1.1 Why Agile?

FinWave adopts the Agile process model with a Scrum-inspired structure adapted to the 24-hour hackathon timeline. Agile was chosen over Waterfall and Spiral for the following key reasons:

- Small team (4–6 members) perfectly suited to Agile's lightweight structure
- Requirements may evolve during the hackathon based on evaluator feedback
- Need to demonstrate a working prototype within 24 hours
- Module-based development aligns with incremental Agile delivery
- Continuous testing and parallel documentation possible

1.2 Agile vs. Alternatives

Criterion	Waterfall	Spiral	Agile (Selected)
Development Style	Sequential, rigid	Risk-driven, complex	Iterative, flexible
Requirement Changes	Very difficult	Moderate	Easily accommodated
Team Size Fit	Large teams	Large, complex	Small teams (3–6)
Delivery Speed	Slow	Moderate	Fast (incremental)
Testing Approach	End of project	Each phase	Continuous / parallel
Hackathon Fit	Poor	Moderate	Excellent
Feedback Integration	Not supported	Limited	Core principle

1.3 Sprint Plan (24-Hour Hackathon)

Sprint	Duration	Goal	Key Deliverables	Owner
Sprint 0	0–1 hr	Planning	Repo setup, scaffolding, task assignment	All
Sprint 1	1–6 hrs	Auth & Wallet	Login/Register, OTP, JWT, Wallet dashboard, DB schema	A + B
Sprint 2	6–14 hrs	Core Features	Transfer form, transaction processing, history, receipts	C + D
Sprint 3	14–20 hrs	Enhancement	i18n (EN/HI/MR), financial tips, UI polish, responsiveness	All
Sprint 4	20–24 hrs	Review & Demo	SRS, UML diagrams, COCOMO, slides, live demo prep	All

1.4 Agile Roles

Role	Member	Responsibilities
Product Owner	Member A	User stories, feature prioritization, MVP alignment

Role	Member	Responsibilities
Scrum Master	Member B	Sprint coordination, blocker removal, standups
Frontend Dev	Member C	React UI, Tailwind CSS, multilingual components
Backend Dev	Member D	Node.js API, Firebase, transaction logic, auth

1.5 Agile Ceremonies

Ceremony	Frequency	Duration	Purpose
Sprint Planning	Before each sprint	15 min	Assign tasks, define sprint goal
Daily Standup	Every 3 hours	5 min	Progress, blockers, plan adjustments
Sprint Review	After each sprint	10 min	Demo features, get feedback
Sprint Retrospective	End of Sprint 2 & 4	5 min	Identify improvements
Backlog Refinement	Sprint 0 + ongoing	10 min	Update priorities based on change

1.6 Product Backlog (Key User Stories)

ID	User Story	Priority	Points	Sprint
US01	Register with mobile number	High	3	Sprint 1
US02	Verify mobile via OTP	High	2	Sprint 1
US03	Secure login	High	3	Sprint 1
US04	View wallet balance	High	2	Sprint 1
US05	Add funds to wallet	Medium	2	Sprint 1
US06	Send money by mobile number	High	5	Sprint 2
US07	Transaction receipt after transfer	High	3	Sprint 2
US08	View transaction history	Medium	3	Sprint 2
US09	Switch app language (Hindi/Marathi)	High	3	Sprint 3
US10	Receive financial tips	Medium	3	Sprint 3
US11	Mobile-friendly interface	High	2	Sprint 3

2. COCOMO Estimation

2.1 What is COCOMO?

COCOMO (Constructive Cost Model) is a software cost estimation model developed by Barry Boehm. It uses Lines of Code (LOC) to estimate effort (person-months), development time (months), and team size. FinWave uses Basic COCOMO in Organic Mode.

2.2 Mode Selection — Organic

FinWave qualifies for Organic Mode based on:

- Small team (6 members) with good domain understanding
- Flexible requirements typical of a startup/hackathon environment
- Estimated codebase of 2,000–3,000 LOC
- Well-understood application domain (fintech web app)
- Relatively relaxed time constraints per module

2.3 LOC Estimation by Module

Module	Components	Est. LOC	Justification
Authentication	Login, Register, OTP	350	Form handling, JWT logic, validation
Wallet	Dashboard, Balance, Add Funds	300	State management, API calls, UI
Send Money	Transfer form, receipt, validation	400	Core business logic, error handling
Transaction History	List, filter, detail view	250	Data fetching, filtering, display
Language (i18n)	EN/HI/MR files + switcher	200	Translation keys, context provider
Financial Tips	Rule engine, tip display	200	Rule conditions, dynamic content
Backend API	Express routes, controllers	600	RESTful endpoints for all modules
Database Models	User, Wallet, Transaction schemas	250	Mongoose/Firebase schema definitions
Config & Misc	Config files, utils, tests	200	Environment setup, helpers
TOTAL	All Modules	2,750	—

2.4 COCOMO Formulae (Organic Mode)

Constants: a = 2.4, b = 1.05, c = 2.5, d = 0.38

Formula	Expression
Effort (E)	E = 2.4 × (KLOC)^1.05 person-months
Development Time (T)	T = 2.5 × (E)^0.38 months

Formula	Expression
Team Size	E / T people

2.5 COCOMO Calculations

Parameter	Value	Formula / Notes
Estimated LOC	2,750	Estimated across all modules
KLOC	2.75	LOC / 1000
Effort (E)	7.11 person-months	2.4 × (2.75)^1.05 = 2.4 × 2.963 = 7.11
Development Time (T)	4.06 months	2.5 × (7.11)^0.38 = 2.5 × 1.624 = 4.06
Recommended Team Size	~2 people	E / T = 7.11 / 4.06 = 1.75 ≈ 2
Productivity	386.8 LOC/person-month	KLOC × 1000 / Effort
Actual Team Size	4 people	Hackathon constraint (faster delivery)
Adjusted Time (4 members)	~1.78 months (~7 days)	7.11 / 4 = 1.78 per person
Hackathon Duration	1 day (24 hours)	Compressed timeline — MVP scope justified

2.6 Justification

The COCOMO estimation validates the project scope. With 4 members, each contributes ~1.78 person-months equivalent — confirming that building an MVP within 24 hours is feasible because:

- A prototype is being built, not production-grade code
- High-productivity frameworks (React, Node.js) significantly accelerate development
- Module responsibilities are cleanly divided among team members
- Documentation, UML, and non-coding deliverables are equally valued by judges

3. UML Design Suite — 14 Diagrams

3.1 Diagram Inventory

#	Diagram Type	Key Focus
01	High-Level Architecture	Modular monolith, cloud tiers, service layers
02	Use Case Diagram	6 actors, include/extend relationships, fintech use cases
03	Class Diagram	Enterprise entities, inheritance, composition, multiplicity
04	Sequence — Money Transfer	ACID transaction safety, atomic wallet ops, rollback handling
05	Activity — Remittance Flow	Decision nodes, failure paths, retry logic, end-to-end
06	State Machine — Transaction	7 states, transition triggers, timeout, fraud path
07	Component Diagram	Layered monolith, interface contracts, loose coupling
08	Deployment Diagram	Vercel + Render + Atlas, CI/CD, cloud-native
09	ER Diagram (MongoDB)	Collections, indexes, relationships, audit trail
10	Data Flow Diagram (L0/L1/L2)	Context DFD, subsystem DFD, process detail DFD
11	Package Diagram	Clean architecture layers, naming conventions
12	Security Architecture	Defence-in-depth, JWT RS256, OWASP Top 10
13	AI Module UML	Cache gateway, LLM adapter, rule-based fallback
14	Blockchain Ledger UML	SHA-256 chain, immutability, verification service

3.2 System Architecture (Diagram 01)

Architecture Tiers

Tier	Technology	Responsibility
Client Tier	React PWA, Tailwind CSS, react-i18next	Mobile-first UI, 10+ languages, offline support
API Gateway	NGINX / Render, express-rate-limit	Rate limiting (100 req/min), CORS policy, JWT pre-filter
Backend App	Node.js / Express.js (Modular Monolith)	Auth, Wallet, Remittance, AI, Blockchain, Compliance modules
Database Tier	MongoDB Atlas (Primary) + Redis (optional)	Users, wallets, transactions, blockchain, audit logs
External Services	Firebase Auth, OpenAI/Gemini, FX Rate API	OTP verification, AI insights, currency conversion

Key Design Decisions

Decision	Justification
Modular Monolith	All modules share one Node.js process — eliminates inter-service latency. Split-ready for microservices post-hackathon.

Decision	Justification
API Gateway Pattern	Centralises JWT validation, rate limiting, and CORS — zero security duplication across services.
MongoDB Atlas	Schema-flexible NoSQL suits rapidly-evolving fintech data models. Free M0 tier covers hackathon.
Firebase Auth (OTP)	Eliminates custom OTP infrastructure. Production-grade SMS delivery globally. Zero setup cost.
AI via REST API	GPT-4o-mini / Gemini called over HTTPS — no ML infrastructure needed. 24h caching prevents rate limit hits.
Stateless JWT Backend	Enables horizontal scaling without sticky sessions. RS256 signing prevents token forgery.

3.3 Use Case Diagram (Diagram 02)

Actors

Actor	Type	Description
Migrant Worker (User)	Primary	Registers, authenticates, manages wallet, sends/receives money, views AI insights
Recipient (Beneficiary)	Passive	Receives in-app/push notification on credit; may or may not be a registered user
System Administrator	Privileged	Manages users, reviews KYC, suspends accounts, views platform analytics
Compliance Officer	Privileged	Reviews fraud alerts, suspicious transactions, KYC document submissions
AI Engine	Automated	Generates categorized insights, savings recommendations, investment nudges
Firebase Auth	External	Handles OTP generation, verification, and Firebase token issuance

Key Use Cases

Use Case	Actor	Includes / Extends
Register (OTP)	Migrant Worker	includes: Firebase OTP Verification
Send Money	Migrant Worker	includes: Validate Balance, Check Recipient, Debit Wallet, Credit Wallet, Generate Hash
View Transaction History	Migrant Worker	extends: Filter by Date/Type/Status, Verify Blockchain Hash
View AI Financial Insights	Migrant Worker	includes: Aggregate Spending; extends: Get Savings Recommendations, Get Investment Nudges
Submit KYC Documents	Migrant Worker	extends: Admin Approval
Search/Filter Users	Admin	—
Review Fraud Alerts	Compliance Officer	extends: Suspicious Txn Review, Approve KYC

3.4 Class Diagram (Diagram 03)

Core Entities

Entity	Key Attributes	Key Methods	Relationships
User	userId (UUID), fullName, email, phoneNumber, role (USER/ADMIN/COMP), kycStatus, preferredLanguage, isActive	register(), login(): JWT, changePassword(), updateProfile()	1 owns 1 Wallet; 1 has 1 KYCRecord; 1 receives 1 AIRecommendation
Wallet	walletId (UUID), userId (FK), balance: Decimal, currency, createdAt	getBalance(), debit(amount), credit(amount), getHistory(): List	1 User owns 1 Wallet; 1 sends/receives * Transactions
Transaction	txnId (UUID), senderWalletId, recipientWalletId, amount, currency, status (INITIATED/PENDING/VALIDATING/PROCESSING/SUCCESS/FAILED/REVERSED), blockchainHash, idempotencyKey	initiate(), validate(): Boolean, process(), reverse()	1 Transaction links to 1 TransactionLedger
TransactionLedger	ledgerId (UUID), blockIndex, transactionId (FK), transactionData (JSON), previousHash, currentHash, nonce, timestamp	computeHash(): String, verify(): Boolean, appendBlock()	1 links to 1 Transaction
KYCRecord	kycId, userId (FK), documentType, documentUrl, verificationStatus, reviewedBy, reviewedAt	submit(), approve(), reject()	1 User has 1 KYCRecord
AIRecommendation	recId, userId (FK), spendingBreakdown (JSON), savingsTarget, investmentSuggestions (JSON), language, cacheExpiresAt	generate(), getCached(), isExpired(): Boolean	1 User receives 1 AIRecommendation

Inheritance Hierarchy

Admin and ComplianceOfficer extend User. All domain entities (User, Wallet, Transaction, TransactionLedger, KYCRecord, AIRecommendation) inherit from the abstract BaseEntity class.

3.5 Sequence Diagram — Money Transfer (Diagram 04)

Happy Path

Step	From	To	Message / Action
1	User	React App	Submit transfer form {recipient, amount, currency}
2	React App	API Gateway	POST /transactions/send (Bearer token)
3	API GW	Auth Svc	verify_JWT → return userId
4	API GW	Wallet Svc	getBalance() → balance=$200
5	API GW	Wallet Svc	[balance >= amt] checkRecipient → recipientFound
6	API GW	Wallet Svc	START MongoDB Session → debit sender
7	API GW	Wallet Svc	credit recipient
8	API GW	Wallet Svc	COMMIT Session

Step	From	To	Message / Action
9	API GW	Transaction Svc	createTxnRecord
10	Transaction Svc	Blockchain Ledger	generateBlock → SHA256 hash
11	Transaction Svc	Notification Svc	notifySender, notifyRecipient
12	React App	User	201 {txnId, hash, status} → success screen

Failure Paths

Scenario	Trigger	System Response
Insufficient Balance	[balance < amt]	ABORT → 400 InsufficientBalance error message
Rollback (Credit Fails)	Recipient credit returns ERROR	ROLLBACK Session (saga) → sender balance restored → 500 TransactionFailed → notifySender(failure)

3.6 State Machine — Transaction Lifecycle (Diagram 06)

Transition	Trigger / Condition
INITIAL → INITIATED	User submits send request; txnId + idempotencyKey set
INITIATED → PENDING	JWT validated, recipient found, transaction object created with UUID
PENDING → VALIDATING	Sufficient wallet balance confirmed; idempotency key unique
VALIDATING → PROCESSING	Optimistic wallet lock acquired; MongoDB session opened
PROCESSING → SUCCESS	Both debit + credit committed; blockchain hash linked
PROCESSING → FAILED (terminal)	Any DB error during session; saga rollback executed
SUCCESS → REVERSED	Reversal request within allowed window (future Phase 2)
FAILED → SUSPICIOUS	Fraud detection anomaly score exceeds threshold
PENDING/VALIDATING → FAILED	Timeout: no response within 30 seconds; lock expires

3.7 Component Diagram (Diagram 07)

Backend Modular Monolith Layers

Layer	Components	Key Operations
Express Router / Middleware	JWTMiddleware, RateLimiterMiddleware, SanitizationMiddleware, ErrorHandlerMiddleware	Auth, throttle, sanitize, error propagation
API Controllers	AuthController, WalletController, TxnController, AIController, AdminController	Request routing, response serialization
Service Layer	AuthService, WalletService, TxnService, AIService, BlockchainSvc, NotificationService, ComplianceService, AuditLogService	Core domain business logic
Repository Layer	UserRepo, WalletRepo, TxnRepo, BlockchainRepo, NotifRepo, AIRepo	MongoDB CRUD via Mongoose ODM

Layer	Components	Key Operations
Database	MongoDB Atlas	Persistent storage for all collections

3.8 Security Architecture (Diagram 11)

Defence-in-Depth Security Layers

Layer	Mechanism	Implementation
Layer 1 – Transport	TLS 1.3, HSTS (max-age=1yr)	Let's Encrypt / Vercel, HTTP → HTTPS redirect (301)
Layer 2 – API Gateway	Rate Limit 100 req/min/IP, CORS Whitelist, Security Headers	express-rate-limit, Helmet.js (CSP, X-Frame, XSS)
Layer 3 – Authentication	Firebase OTP + JWT RS256	Access token 15min, refresh 7d, account lock 5 fails → 15min ban
Layer 4 – Authorization	RBAC roleGuard middleware	USER role: wallet/txn/insights; ADMIN: + user mgmt; COMPLIANCE: + KYC/fraud
Layer 5 – Input Validation	Zod/Joi schema, mongo-sanitize, xss-clean	req body validation, strips $/.operators, HTML escape
Layer 6 – Data Security	bcrypt(12) passwords, AES-256-GCM KYC docs, RS256 JWT	Asymmetric signing, encrypted sensitive fields
Layer 7 – Audit	AuditLog collection, 90-day TTL retention	All auth events logged with IP; auto-flag suspicious transactions

OWASP Top 10 Mitigations

OWASP Risk	FinWave Mitigation
A01 – Broken Access Control	RBAC roleGuard on every route; 403 on unauthorized access; admin routes prefixed /admin
A02 – Cryptographic Failures	TLS 1.3 everywhere; bcrypt(12) passwords; AES-256 KYC docs; RS256 JWT
A03 – Injection	mongo-sanitize; Mongoose parameterized queries; Zod schema validation
A05 – Security Misconfiguration	Helmet.js headers; CORS whitelist; env vars in Render secrets
A07 – Authentication Failures	Account lockout (5 attempts); OTP expiry (5min); JWT short expiry (15min)
A09 – Logging Failures	AuditLog collection; all auth events recorded with IP; 90-day TTL retention

3.9 AI Module Design (Diagram 13)

AI Module Flow

Component	Role	Details
CacheGateway	Cache check	Checks AI_Insights collection for userId with expiresAt > now; returns HIT or MISS

Component	Role	Details
TransactionAggregator	Data prep	Queries last 90 days txns, groups by 8 categories, computes monthly totals & trends, strips all PII
PromptBuilder	Prompt construction	Structured prompt in user's language; injects category totals, balance trend, goals, lang
LLMAdapter	AI call	POST to OpenAI GPT-4o-mini / Gemini Flash; retries 2x on timeout; falls back to RuleEngine
ResponseParser	Response processing	Validates JSON schema; extracts spendingBreakdown, savingsTip, investmentSuggestions, riskScore
CacheWriter	Cache write	Stores insights to AI_Insights collection; sets expiresAt = now + 24h; TTL index auto-cleans
RuleEngine (Fallback)	Fallback logic	Pre-defined tips: spending >40% food; remittance >50% income; balance <$50

3.10 Blockchain Ledger UML (Diagram 14)

Block Structure

Field	Type	Purpose
blockIndex	Integer	Sequential block number (Genesis = 0)
timestamp	ISO 8601 UTC	Block creation time
transactionData	JSON	Sender, receiver, amount, currency, transactionId
previousHash	String	SHA-256 hash of previous block (chain linkage)
currentHash	String	SHA-256 of blockIndex + timestamp + sortedData + previousHash + nonce
nonce	Number	Simulates proof-of-work; incremented until hash meets criteria

Hash Computation: hash = crypto.createHash('sha256').update(JSON.stringify(blockData, Object.keys(blockData).sort())).digest('hex')

Design Justification

Design Choice	Justification
SHA-256 via Node.js crypto	Built-in, zero dependencies, same algorithm as Bitcoin — cryptographically sound for simulation
MongoDB for ledger storage	Indexed on blockIndex for O(log n) chain traversal. TTL-free (blocks are permanent)
Sorted JSON serialization	Prevents hash inconsistency from JS object key ordering — deterministic hash for same data
Chain linkage (previousHash)	If any block is tampered, all subsequent hashes become invalid — demonstrates immutability
Nonce field	Simulates proof-of-work concept for educational value

4. Database Design (MongoDB Atlas)

4.1 Collections Overview

Collection	Primary Key	Description	Key Indexes
users	_id (ObjectId)	User profiles, roles, KYC status, language preference	firebaseUID (unique), email (unique), phoneNumber (unique)
wallets	_id (ObjectId)	Digital wallets linked to users; balance in Decimal128	userId (unique FK), walletId (unique)
transactions	_id (ObjectId)	All money transfers; atomic debit/credit pairs	senderWalletId, recipientWalletId, idempotencyKey (unique), createdAt
blockchain_ledger	_id (ObjectId)	Simulated blockchain blocks with SHA-256 hashing	blockIndex (unique), transactionId (FK)
ai_insights	_id (ObjectId)	Cached AI-generated financial insights (24h TTL)	userId, expiresAt (TTL index)
notifications	_id (ObjectId)	In-app and push notification records	userId, createdAt; 30-day TTL
kyc_records	_id (ObjectId)	KYC document references and verification status	userId (unique FK)
audit_logs	_id (ObjectId)	All authentication and admin action audit trail	userId, timestamp; 90-day TTL

4.2 Performance-Critical Indexes

Index	Collection	Type	Purpose
firebaseUID	users	Unique	Firebase token verification lookup
senderWalletId + recipientWalletId	transactions	Compound	Transaction history queries
idempotencyKey	transactions	Unique	Prevent duplicate transaction processing
expiresAt	ai_insights	TTL (24h)	Auto-expiry of cached AI insights
blockIndex	blockchain_ledger	Unique	O(log n) chain traversal
createdAt (notifications)	notifications	TTL (30 days)	Auto-cleanup old notifications
timestamp (audit_logs)	audit_logs	TTL (90 days)	Compliance retention and auto-cleanup

5. API Design

All endpoints prefixed: /api/v1/ | Format: JSON | Auth: JWT Bearer token (except /auth/register and /auth/login)

5.1 Authentication Endpoints

Method	Endpoint	Description	Status Codes
POST	/auth/register	Register new user; Firebase OTP must be verified first	201, 400, 409, 401
POST	/auth/login	Authenticate existing user via Firebase token	200, 401, 403, 429
POST	/auth/refresh	Issue new access token using valid refresh token	200, 401
POST	/auth/logout	Invalidate all active sessions for the user	200, 401

5.2 Wallet Endpoints

Method	Endpoint	Description	Status Codes
GET	/wallet/balance	Fetch authenticated user's wallet balance and currency	200, 401, 404
POST	/wallet/add-funds	Add simulated funds to wallet (prototype only)	200, 400, 401
GET	/wallet/summary	Wallet overview with recent 5 transactions	200, 401

5.3 Transaction Endpoints

Method	Endpoint	Description	Status Codes
POST	/transactions/send	Initiate zero-fee money transfer (atomic debit/credit)	201, 400, 402, 404, 422
GET	/transactions/history	Paginated transaction history with optional filters	200, 401, 422
GET	/transactions/:id	Fetch single transaction detail with blockchain hash	200, 401, 404
GET	/blockchain/verify/:hash	Verify transaction integrity via blockchain hash lookup	200, 404

5.4 AI & User Endpoints

Method	Endpoint	Description	Status Codes
POST	/ai/insights	Generate or return cached AI financial insights	200, 401, 503

Method	Endpoint	Description	Status Codes
GET	/user/profile	Fetch authenticated user's profile data	200, 401
PUT	/user/language	Update preferred language (takes effect immediately)	200, 400, 401
PUT	/user/profile	Update name, profile picture	200, 400, 401
GET	/notifications	Fetch paginated notification history	200, 401

6. Non-Functional Requirements

6.1 Performance

Requirement	Target / Metric
API Response Time (P95)	< 500ms under normal load (< 100 concurrent users)
Page Load Time (Initial)	< 3 seconds on 4G connection
Transaction Processing Time	< 2 seconds for domestic transfers
AI Insight Generation	< 8 seconds (includes OpenAI/Gemini API call)
Dashboard Auto-Refresh	Every 60 seconds
Database Query Time	< 100ms for indexed queries

6.2 Availability SLA

Component	Target SLA	Failover Strategy
Frontend (CDN)	99.99%	Global CDN with edge failover
Backend API	99.5%	Auto-restart on failure, health checks at /api/health
MongoDB Atlas	99.995% (M10+)	Replica set with automatic failover
Firebase Auth	99.95%	Firebase SLA guarantee
AI API (OpenAI)	99.9%	24h cached responses, rule-based graceful degradation

6.3 Security NFRs

- All data in transit encrypted via TLS 1.3 (minimum TLS 1.2)
- Passwords hashed using bcrypt with salt factor = 12
- Sensitive fields (KYC docs, bank account numbers) encrypted at rest with AES-256-GCM
- JWT tokens use RS256 asymmetric signing; private key stored server-side only
- OWASP Top 10 security controls implemented throughout the stack
- API rate limiting: 100 requests/minute per IP via express-rate-limit

6.4 Usability

- First-time users can complete registration in under 3 minutes
- Core task flows (send money, view balance) require no more than 3 taps/clicks
- WCAG 2.1 Level AA accessibility compliance throughout all screens
- Screen reader compatible (ARIA labels on all interactive elements)
- Support for screen sizes from 320px (small mobile) to 1920px (desktop)

6.5 Scalability

- Stateless backend enables horizontal scaling via container orchestration

- MongoDB Atlas scales vertically (M0 → M10+) without application code changes
- Frontend CDN handles 100,000+ concurrent requests via global edge caching
- API versioning (/api/v1/) supports future breaking changes without disrupting clients

7. Future Scope

Feature	Phase	Description
Real Banking Integration	Phase 2	Connect to Plaid / Open Banking UK for real account-to-account transfers
UPI Integration (India)	Phase 2	Integrate with NPCI's UPI rails for zero-fee INR transfers
Real Blockchain (Polygon)	Phase 3	Replace simulated ledger with Polygon PoS for true immutable records
International Remittance Rails	Phase 2	Partner with dLocal / Thunes for real cross-border settlement
Micro-Insurance Products	Phase 3	Parametric insurance (health, travel, income protection) via API partners
Micro-Loan System	Phase 3	AI credit scoring-based micro-loans ($10–$500) repaid via wallet
AI Credit Scoring	Phase 3	ML model trained on transaction behavior for alternative credit scores
Voice Assistant	Phase 3	Multilingual voice interface for balance queries and send-money commands
Offline Mode	Phase 2	PWA offline capability with sync queue for low-connectivity areas
Group Savings (Chama)	Phase 3	Digital ROSCA — group savings pools popular in African and South Asian communities

8. Prototype Limitations & Mitigations

Limitation	Impact	Mitigation Path
Simulated Transactions	No real money moves; balance updates in MongoDB only	Integrate licensed payment processor (Phase 2)
Simulated Blockchain	Not a real distributed ledger; security is illustrative	Migrate to Polygon or Hyperledger Fabric (Phase 3)
No Real KYC	Identity verification cosmetic; AML/KYC not implemented	Integrate Jumio or Onfido (Phase 2)
AI Rate Limits	OpenAI/Gemini API throttling under high demo load	Implement request queuing and improved caching
Limited Currencies	Only 5 currencies; FX rates are simulated, not live	Integrate Open Exchange Rates API (Phase 2)
Free-Tier Backend	Render.com sleeps after 15 min; cold start ~30s	Upgrade to paid instance for production
No Offline Mode	Application requires internet for all operations	Implement Service Worker cache strategies (Phase 2)
Limited Test Coverage	Hackathon timeline limits test coverage below 70%	Expand test suite post-hackathon

9. UI/UX Design System

Element	Specification	Rationale
Primary Color	#1B4F72 (Deep Navy Blue)	Trust, financial stability
Accent Color	#1ABC9C (Teal)	Action, success, positive sentiment
Typography	Inter (UI), Noto Sans (Multilingual fallback)	Both Google Fonts; Noto supports all 10+ languages
Border Radius	12px cards, 8px inputs, 24px primary buttons	Friendly but professional feel
Spacing System	4px base unit (multiples: 8, 12, 16, 24, 32, 48px)	Consistent visual rhythm
Shadow	0 2px 8px rgba(0,0,0,0.08)	Subtle depth without heaviness
Icon Library	Lucide React	Open-source, tree-shakeable, consistent stroke width
Accessibility	WCAG 2.1 AA, 4.5:1 contrast ratio, 44x44px touch targets	Designed for low-literacy migrant worker users